

MSc PROJECT DEFINITION 2025-26

School of Electronic Engineering and Computer Science Queen Mary University of London

Project Title: Auto-Echo: A Framework for Automated Discovery of Memory Hierarchy Latency Patterns

Supervisor: Vasileios Klimis **Student name:** Harsh Raj Singh **Student e-mail:** ec25303@qmul.ac.uk

1. PROJECT AIMS

The main goal of this project is to build an automated tool—which I'm calling "Auto-Echo"—that can figure out the structure of a computer's memory hierarchy without needing to look up the manufacturer's spec sheet. It works by collecting "echolocation" data (essentially timing how long memory accesses take) and then using unsupervised machine learning to group those timings. These groups map directly to the different cache levels like L1, L2, L3, and the main DRAM.

Right now, figuring out memory access latency usually means looking at timing plots manually and guessing where the architectural boundaries lie based on visual jumps in the data. This project wants to change that manual guesswork into a scalable, automated process. By writing probes that run natively and measure access timings, we can abstract away the complex proprietary hardware and just look at the statistical clusters. Ultimately, I want to create a software framework that works across different types of hardware, making low-level system profiling much more straightforward and self-calibrating.

2. PROJECT OBJECTIVES

To make the "Auto-Echo" framework a reality, I've broken the project down into several key objectives:

- 1. Build the Hardware Probe:** I'll need to write a reliable memory echolocation probe that can accurately time how long memory accesses take. This probe needs to gather a large enough dataset of access latencies across all the cache layers, while avoiding background noise from the operating system as much as possible.
 - 2. Filter Out the Noise:** The raw timing data from hardware is never clean; it's full of noise from context switches, varying power states, and interrupts. I plan to use statistical filtering—like removing extreme outliers and using moving averages—to clean the data before feeding it into any models.
 - 3. Integrate Machine Learning:** This is where the automation comes in. I'll integrate unsupervised machine learning algorithms (specifically K-Means Clustering, Gaussian Mixture Models, and DBSCAN) so the framework can analyze the timing arrays mathematically and find the natural latency boundaries.
 - 4. Automate the Model Selection:** I don't want the user to have to guess how many cache levels exist beforehand. So, a major objective is to write the logic (using methods like the Silhouette Score or the Elbow Method) to let the platform automatically figure out the correct number of memory levels (the 'k' clusters).
 - 5. Validate Against Real Hardware:** Once the framework outputs a predicted architecture, I need to know it's right. I will validate the program's output by running it on varied machines (like modern x86 computers and ARM-based single-board computers) and comparing my results against the official hardware specifications.
 - 6. Align with MSc Outcomes:** This project is a great fit for the Computer Science MSc because it bridges low-level systems programming (the data collection) with applied data science (the unsupervised learning component). It's a practical, real-world application of both skill sets.
-

3. METHODOLOGY AND TECHNICAL APPROACH

To keep things organized and ensure I don't get stuck, the project is structured into overlapping phases. It starts at the systems level and builds up to the machine learning abstraction.

3.1 Background Literature and Theory

I'll start by reading up on memory echolocation algorithms, taking heavy inspiration from papers like *"Shouting at Memory: Where Did My Write Go?"* alongside researching how to apply unsupervised clustering to single-dimensional timing data.

3.2 Probe Engineering and Data Collection

Writing the timing probe is going to require some low-level C or C++ programming. I'll likely use inline assembly instructions to get the most accurate cycle counts (for example, `rdtscp` on Intel or the equivalent cycle counters on ARM). The data collection will involve running loops over different memory stride lengths and recording millions of individual timings. To get the cleanest data, I'll explore techniques like pinning threads to specific CPU cores.

3.3 Data Pre-processing Pipeline

Once I have the raw latency CSVs, the data needs to be shaped. I'm going to use Python and the Pandas library here. I'll write scripts to toss out the statistical outliers using Interquartile Range (IQR) techniques. This is a vital step—if context-switch latency spikes make it into the training data, they will throw off the cluster centers entirely.

3.4 Unsupervised Machine Learning

The cleaned data will then be fed into Python's `scikit-learn` models:

- **K-Means Clustering:** I'll use this as the baseline to see if it can easily find distinct latency bands.
- **Gaussian Mixture Models (GMM):** Since cache latencies can blur together where one cache level evicts to the next, GMM might handle the "soft" boundaries better than K-Means.
- **DBSCAN:** I'm also planning to test DBSCAN because it doesn't assume clusters are spherical, which might help if main-memory latency events are scattered.

3.5 Dynamic Architecture Abstraction

The script needs to figure out how many levels of cache there are on its own. It will do this by iterating through possible depths (say, 1 to 10 levels) and plotting the Silhouette Coefficients to find the highest confidence mapping.

3.6 Evaluation and Validation

To prove "Auto-Echo" works, I'll run controlled experiments on lab machines where the exact hardware specs are a known quantity (like an Intel Core i7). The framework's terminal output needs to match up with the machine's actual cache layout.

4. PROJECT MILESTONES AND DELIVERABLES

I've set out the following measurable milestones for the project:

- **Milestone 1:** The echolocation probe is written in C/C++, runs on a baseline test machine, and successfully outputs raw latency datasets into CSV files.
 - **Milestone 2:** The Python data-cleaning pipeline is finished. I should be able to produce "before and after" histograms showing that extreme outliers and OS noise have been successfully stripped.
 - **Milestone 3:** The unsupervised clustering models are integrated into the pipeline, and they successfully group the latency points when I tell them the correct 'k' value manually.
 - **Milestone 4:** The automated selection routine is complete. The system can now analyze a dataset and programmatically guess the optimal topology without any manual input.
 - **Milestone 5:** The final `Auto-Echo` framework is complete and usable via a Command Line Interface (CLI).
 - **Milestone 6:** The final validation report is written, documenting the framework's error rate when run against a variety of different processor architectures.
-

5. REQUIRED KNOWLEDGE, SKILLS, TOOLS, AND RESOURCES

Successfully finishing this project requires applying a mix of specific skills and tools:

- **Low-Level Systems Programming:** I need a strong grasp of C/C++ memory management and inline assembly (for CPU cycle counting), as well as an understanding of how caches and OS threading work.
- **Data Engineering:** Good practical knowledge of Python (specifically Pandas and Numpy) to manipulate arrays containing millions of data points quickly.
- **Data Science / Machine Learning:** A solid grasp of how to practically apply `scikit-learn` models, understand vector spaces, and evaluate model performance using statistics like the Silhouette score.
- **Development Tooling:** GCC, Makefiles, VSCode, and Git for keeping track of all the code versions.
- **Hardware Resources:** Access to dedicated laboratory test machines (ideally crossing architecture types, like x86 and ARM) so I can ensure the results translate across different hardware.

6. PROJECT PLAN AND TIMEPLAN (GANTT BREAKDOWN)

The project will take place over roughly 16 weeks. I've broken the timeline down to make sure there's enough room for both the coding and the final academic write-up.

Phase 1: Research and Probing (Weeks 1-4)

- **Week 1-2:** Deep dive into the literature around cache echolocation. Getting familiar with the theoretical side of clustering timing data.
- **Week 3:** Developing the baseline C/C++ memory probe.
- **Week 4:** Initial testing on a local machine to extract the first massive multi-megabyte datasets.

Phase 2: Data Pipeline & Cleaning (Weeks 5-7)

- **Week 5:** Bootstrapping the Python pipeline. Writing scripts to ingest the CSV files and plotting early histograms to visualize the data shape.
- **Week 6:** Programming the outlier rejection logic so the OS noise is cleanly removed.
- **Week 7:** Stabilizing the dataset. Doing manual checks to see if the cleaned data looks like it maps to the expected hardware topology.

Phase 3: Machine Learning Model Development (Weeks 8-11)

- **Week 8:** Firing up Scikit-Learn and applying K-Means clustering to the pipeline.
- **Week 9:** Experimenting with Gaussian Mixture Models (GMM) to calculate probabilities across the latency map.
- **Week 10:** Writing the automation logic so the system can use Silhouette scoring to find the right number of clusters dynamically.
- **Week 11:** Integrating the clustering module permanently into the main `Auto-Echo` codebase.

Phase 4: Validation, Refinement, and Reporting (Weeks 12-16)

- **Week 12:** Running the full framework on secondary machines with varying cache architectures to test its limits.
- **Week 13:** Buffer week. Optimising any slow code in the pipeline, fixing bugs, and writing simple guidelines on how to run the CLI tool.
- **Week 14:** Focused work on structuring and drafting the main body of the MSc project report.
- **Week 15:** Writing up the methodology, validation results, and engineering challenges into the report.
- **Week 16:** Final proofreading, structuring the dissertation format, organizing the codebase repository, and getting the final package ready for submission.